UD3.2: Generadores y funciones avanzadas

Módulo: ECP — Estructuras de Control en Python (5099)
Unidad: UD3 — Programación Avanzada
Contenido: C7 (Generadores) · C8 (Lambda, map, filter, recursión)

Índice de contenidos

1. Generadores
2. Funciones lambda
3. map() y filter()
4. Funciones recursivas
5. Ejercicios integradores (resueltos)
6. Tabla resumen

1. Generadores

> **Criterios**: CE1_7 (bloques de control de repetición), CE3_4 (sentencias iterativas según necesidades)

1.1. El problema: listas grandes en memoria

En el tema anterior se vieron las list comprehensions:

```
cuadrados = [x ** 2 for x in range(1_000_000)]
```

Esta línea crea una lista de **un millón de elementos en memoria**. Si solo se necesita recorrerlos uno a uno, se está ocupando memoria innecesariamente.

1.2. Expresiones generadoras

Un **generador** produce los elementos **uno a uno**, bajo demanda, sin almacenarlos todos en memoria. La sintaxis es idéntica a una list comprehension, pero con **paréntesis** en vez de corchetes:

```
# List comprehension → crea toda la lista en memoria
cuadrados_lista = [x ** 2 for x in range(1_000_000)]

# Generador → produce los valores uno a uno
cuadrados_gen = (x ** 2 for x in range(1_000_000))


print(type(cuadrados_lista))  # <class 'list'>
print(type(cuadrados_gen))    # <class 'generator'>
```

Un generador es **iterable**: se puede usar en un `for` igual que una lista:

```
gen = (x ** 2 for x in range(5))

for valor in gen:
    print(valor)
# 0, 1, 4, 9, 16
```

1.3. next() y agotamiento

Los elementos de un generador se pueden obtener uno a uno con `next()` :

```
gen = (x ** 2 for x in range(3))

print(next(gen))  # 0
print(next(gen))  # 1
print(next(gen))  # 4
print(next(gen))  # StopIteration (error: no quedan elementos)
```

Diferencia fundamental con una lista: un generador **se agota**. Una vez consumidos todos los elementos, no se puede volver atrás.

```python
gen = (x for x in range(3))

# Primera pasada: funciona
for x in gen:
    print(x)  # 0, 1, 2

# Segunda pasada: no imprime nada (ya está agotado)
for x in gen:
    print(x)  # (nada)
```

1.4. Funciones generadoras (yield)

Se pueden crear generadores con funciones que usen `yield` en vez de `return`:

```python
def contar_hasta(n):
    """Genera números del 1 a n."""
    i = 1
    while i <= n:
        yield i
        i += 1

for num in contar_hasta(5):
    print(num)
# 1, 2, 3, 4, 5
```

Diferencia entre `yield` y `return`:

`return`	`yield`
La función **termina**	La función **se pausa**
Devuelve un valor y se destruye	Devuelve un valor y recuerda dónde se quedó
Se ejecuta una sola vez	Se reanuda en cada llamada a `next()`

```python
def pares_hasta(n):
    """Genera los números pares hasta n."""
    for i in range(2, n + 1, 2):
        yield i

list(pares_hasta(10))  # [2, 4, 6, 8, 10]
```

> **Nota**: `list()` convierte un generador en una lista (consumiéndolo). Es útil para ver todos los valores, pero anula la ventaja de memoria.

1.5. Cuándo usar generadores vs listas

Situación	Mejor opción
Se necesita acceder a los elementos varias veces	Lista
Se necesita acceso por índice (`lista[3]`)	Lista
Se necesita `len()` , `sort()` , `reverse()`	Lista
La secuencia es muy grande	Generador
Solo se recorre una vez (en un `for` o `sum()`)	Generador

Regla práctica: si la lista cabe en memoria y se va a usar más de una vez, usar lista. Si es muy grande o solo se recorre una vez, usar generador.

1.6. Ejercicios

Ejercicio G1: Convertir a generador

Dada la siguiente list comprehension, convertirla en una expresión generadora y sumar sus valores con `sum()` :

```
cuadrados = [x ** 2 for x in range(1, 101)]
total = sum(cuadrados)
print(total)
# Resultado esperado: 338350
```

> **Nota**: `sum()` , `min()` , `max()` y `list()` aceptan generadores directamente.

Solución:

```
total = sum(x ** 2 for x in range(1, 101))
print(total)  # 338350
```

> Cuando el generador es el **único argumento** de una función, se pueden omitir los paréntesis exteriores: `sum(x**2 for x in ...)` en vez de `sum((x**2 for x in ...))` .

Ejercicio G2: Generador con yield

Escribir una función generadora `fibonacci(n)` que produzca los primeros `n` números de Fibonacci.

```
fibonacci(8) → 0, 1, 1, 2, 3, 5, 8, 13
```

Solución:

```python
def fibonacci(n):
    a, b = 0, 1
    for _ in range(n):
        yield a
        a, b = b, a + b

print(list(fibonacci(8)))  # [0, 1, 1, 2, 3, 5, 8, 13]
```

Ejercicio G3: Generador con filtro

Escribir una función generadora `aprobados(notas)` que reciba una lista de tuplas `(nombre, nota)` y produzca solo los nombres de los que tienen nota >= 5.

```
datos = [("Ana", 7), ("Luis", 3), ("Marta", 9), ("Pedro", 4)]
# aprobados(datos) → "Ana", "Marta"
```

Solución:

```python
def aprobados(notas):
    for nombre, nota in notas:
        if nota >= 5:
            yield nombre

datos = [("Ana", 7), ("Luis", 3), ("Marta", 9), ("Pedro", 4)]
print(list(aprobados(datos)))  # ['Ana', 'Marta']
```

2. Funciones lambda

> **Criterio**: CE4_4 (funciones lambda en un programa de Python)

2.1. Qué son

Una **función lambda** es una función anónima (sin nombre) de **una sola expresión**:

```
# Función normal
def doble(x):
    return x * 2

# Equivalente con lambda
doble = lambda x: x * 2

print(doble(5))  # 10
```

Sintaxis:

```
lambda parámetros: expresión
```

- No lleva `def` ni nombre
- No lleva `return` (el resultado se devuelve automáticamente)
- Solo puede contener **una expresión** (no varias líneas)

2.2. Cuándo usarlas

Las funciones lambda son útiles como **funciones cortas de usar y tirar**, especialmente como argumento de otras funciones:

```
# Sin lambda: definir una función aparte
def es_par(x):
    return x % 2 == 0

pares = filter(es_par, range(10))

# Con lambda: en el mismo sitio
pares = filter(lambda x: x % 2 == 0, range(10))
```

2.3. Ejemplos

```python
# Lambda con un parámetro
cuadrado = lambda x: x ** 2
print(cuadrado(4))  # 16

# Lambda con dos parámetros
suma = lambda a, b: a + b
print(suma(3, 5))  # 8

# Lambda con expresión condicional
clasificar = lambda x: "par" if x % 2 == 0 else "impar"
print(clasificar(7))  # "impar"
```

2.4. Uso principal: sorted con key

El uso más habitual de lambda es como argumento `key` de `sorted()` :

```python
alumnos = [("Ana", 7), ("Luis", 9), ("Marta", 5)]

# Ordenar por nota (segundo elemento de cada tupla)
por_nota = sorted(alumnos, key=lambda a: a[1])
# [('Marta', 5), ('Ana', 7), ('Luis', 9)]

# Ordenar por nota de mayor a menor
por_nota_desc = sorted(alumnos, key=lambda a: a[1], reverse=True)
# [('Luis', 9), ('Ana', 7), ('Marta', 5)]
```

2.5. Cuándo NO usarlas

```python
# ❌ MAL: lambda asignada a variable con nombre → usar def
multiplicar = lambda x, y: x * y

# ✅ BIEN: si necesita un nombre, mejor una función
def multiplicar(x, y):
    return x * y
```

Regla: si una lambda necesita un nombre, usar `def` . Las lambdas son para funciones que se usan una sola vez, como argumento de `sorted()` , `map()` o `filter()` .

2.6. Ejercicios

Ejercicio L1: Ordenar con lambda

Dada `palabras = ["python", "es", "un", "lenguaje"]` , ordenarlas por longitud (más cortas primero) usando `sorted()` con `key=lambda` .

```
Resultado esperado: ['es', 'un', 'python', 'lenguaje']
```

Solución:

```python
palabras = ["python", "es", "un", "lenguaje"]
resultado = sorted(palabras, key=lambda p: len(p))
print(resultado)  # ['es', 'un', 'python', 'lenguaje']
```

Ejercicio L2: Lambda con expresión condicional

Crear una lambda que reciba un número y devuelva `"aprobado"` si es >= 5, o `"suspenso"` en caso contrario. Probarla con `[3, 7, 4, 9, 5]` .

```
Resultado esperado: ['suspenso', 'aprobado', 'suspenso', 'aprobado', 'aprobado']
```

Solución:

```python
evaluar = lambda n: "aprobado" if n >= 5 else "suspenso"

notas = [3, 7, 4, 9, 5]
resultados = [evaluar(n) for n in notas]
print(resultados)
# ['suspenso', 'aprobado', 'suspenso', 'aprobado', 'aprobado']
```

Ejercicio L3: Ordenar con criterio complejo

Dada `palabras = ["Python", "es", "un", "lenguaje", "de", "programación"]` , ordenarla por longitud y, en caso de empate, alfabéticamente.

```
Resultado esperado: ['de', 'es', 'un', 'Python', 'lenguaje', 'programación']
```

Solución:

```python
palabras = ["Python", "es", "un", "lenguaje", "de", "programación"]
resultado = sorted(palabras, key=lambda p: (len(p), p.lower()))
print(resultado)
# ['de', 'es', 'un', 'Python', 'lenguaje', 'programación']
```

> La clave devuelve una **tupla**. Python compara tuplas elemento a elemento: primero por longitud, y en caso de empate, alfabéticamente.

3. map() y filter()

> **Criterio**: CE4_2 (funciones built-in de Python)

3.1. map(): transformar cada elemento

`map(función, iterable)` aplica una función a **cada elemento** y devuelve un iterador:

```python
numeros = [1, 2, 3, 4, 5]

# Con map + lambda
dobles = list(map(lambda x: x * 2, numeros))
# [2, 4, 6, 8, 10]

# Equivalente con comprehension
dobles = [x * 2 for x in numeros]
# [2, 4, 6, 8, 10]
```

Con una función ya existente, no hace falta lambda:

```python
numeros_str = ["3", "7", "10"]
numeros = list(map(int, numeros_str))
# [3, 7, 10]
```

3.2. filter(): seleccionar elementos

`filter(función, iterable)` selecciona los elementos para los que la función devuelve `True`:

```
numeros = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

# Con filter + lambda
pares = list(filter(lambda x: x % 2 == 0, numeros))
# [2, 4, 6, 8, 10]

# Equivalente con comprehension
pares = [x for x in numeros if x % 2 == 0]
# [2, 4, 6, 8, 10]
```

3.3. Equivalencias con comprehensions

Operación	map/filter	Comprehension
Transformar	`list(map(f, seq))`	`[f(x) for x in seq]`
Filtrar	`list(filter(f, seq))`	`[x for x in seq if f(x)]`
Ambas	`list(map(f, filter(g, seq)))`	`[f(x) for x in seq if g(x)]`

> **En Python moderno**, las comprehensions son la opción preferida en la mayoría de casos por ser más legibles. `map` y `filter` son útiles cuando ya existe una función con nombre (`map(int, lista_de_strings)`).

3.4. Ejercicios

Ejercicio M1: map con función existente

Dada `numeros_str = ["3", "7", "-2", "10", "0"]`, usar `map` para convertirlos a enteros y calcular su suma.

```
Resultado esperado: 18
```

Solución:

```python
numeros_str = ["3", "7", "-2", "10", "0"]
total = sum(map(int, numeros_str))
print(total)  # 18
```

Ejercicio M2: filter con lambda

Dada `palabras = ["hola", "", "mundo", "", "python", ""]`, usar `filter` para eliminar las cadenas vacías.

```
Resultado esperado: ['hola', 'mundo', 'python']
```

Solución:

```python
palabras = ["hola", "", "mundo", "", "python", ""]

# Opción 1: filter con lambda
resultado = list(filter(lambda p: p != "", palabras))

# Opción 2: filter con None (filtra valores "falsy")
resultado = list(filter(None, palabras))

print(resultado)  # ['hola', 'mundo', 'python']
```

Ejercicio M3: map + filter combinados

Dada una lista de notas como strings, obtener las notas aprobadas (>= 5) como enteros:

```python
notas_str = ["3.5", "7.2", "4.8", "9.0", "5.0"]
```

Resultado esperado: `[7, 5, 9, 5]`

Solución:

```python
notas_str = ["3.5", "7.2", "4.8", "9.0", "5.0"]

# Con map + filter
notas = map(float, notas_str)
aprobadas = filter(lambda n: n >= 5, notas)
resultado = list(map(round, aprobadas))
# [7, 5, 9, 5]

# Equivalente con comprehension
resultado = [round(float(n)) for n in notas_str if float(n) >= 5]
```

4. Funciones recursivas

> **Criterio**: CE4_5 (funciones recursivas partiendo de funciones definidas anteriormente)

4.1. Qué es la recursión

Una función **recursiva** es una función que **se llama a sí misma** para resolver un problema dividiéndolo en subproblemas más pequeños.

Toda función recursiva necesita:

1. **Caso base**: condición que detiene la recursión (sin él, la función se llamaría infinitamente)
2. **Caso recursivo**: la función se llama a sí misma con un problema **más pequeño**

4.2. Ejemplo: factorial

El factorial de un número `n` (escrito `n!`) es el producto de todos los enteros desde 1 hasta `n`:

```
5! = 5 × 4 × 3 × 2 × 1 = 120
```

Se puede definir recursivamente: `n! = n × (n-1)!`, con el caso base `0! = 1`.

```python
# Versión iterativa (con bucle)
def factorial_iterativo(n):
    resultado = 1
    for i in range(1, n + 1):
        resultado *= i
    return resultado

# Versión recursiva
def factorial(n):
    if n == 0:           # Caso base
        return 1
    return n * factorial(n - 1)  # Caso recursivo

print(factorial(5))  # 120
```

Cómo funciona `factorial(5)`:

```
factorial(5)
  → 5 * factorial(4)
    → 4 * factorial(3)
      → 3 * factorial(2)
        → 2 * factorial(1)
          → 1 * factorial(0)
            → 1              ← caso base
          → 1 * 1 = 1
        → 2 * 1 = 2
      → 3 * 2 = 6
    → 4 * 6 = 24
  → 5 * 24 = 120
```

4.3. Ejemplo: suma de una lista

Sumar todos los elementos de una lista sin usar `sum()` ni bucles:

```python
def suma_lista(lista):
    if len(lista) == 0:           # Caso base: lista vacía
        return 0
    return lista[0] + suma_lista(lista[1:])  # Primer elemento + suma del resto

print(suma_lista([3, 1, 4, 1, 5]))  # 14
```

4.4. Ejemplo: cuenta atrás

```
def cuenta_atras(n):
    if n <= 0:           # Caso base
        print("¡Despegue!")
        return
    print(n)
    cuenta_atras(n - 1)  # Caso recursivo

cuenta_atras(3)
# 3
# 2
# 1
# ¡Despegue!
```

4.5. Errores comunes

Olvidar el caso base → recursión infinita:

```
# ❌ Sin caso base → RecursionError: maximum recursion depth exceeded
def factorial_mal(n):
    return n * factorial_mal(n - 1)  # Nunca se detiene
```

Caso base incorrecto → no cubre todos los casos:

```
# ❌ No funciona con n=0
def factorial_mal(n):
    if n == 1:
        return 1
    return n * factorial_mal(n - 1)

factorial_mal(0)  # RecursionError (nunca llega a n==1)
```

4.6. Recursión vs iteración

Aspecto	Recursión	Iteración (bucle)
Legibilidad	Elegante para problemas naturalmente recursivos	Más directa para problemas lineales

Aspecto	Recursión	Iteración (bucle)
Rendimiento	Más lenta (cada llamada crea un nuevo contexto)	Más rápida
Memoria	Usa más (pila de llamadas)	Usa menos
Límite	Python limita a ~1000 llamadas recursivas	Sin límite
Mejor para	Árboles, fractales, divide y vencerás	Recorrer listas, acumular valores

Regla práctica: si el problema se describe naturalmente como "resolver el mismo problema pero más pequeño", la recursión encaja bien. Si no, un bucle suele ser mejor opción.

4.7. Ejercicios

Ejercicio R1: Potencia recursiva

Escribir una función recursiva `potencia(base, exponente)` que calcule `base^exponente` sin usar `**`.

```
potencia(2, 3) → 8   (2 × 2 × 2)
potencia(5, 0) → 1   (caso base: cualquier número elevado a 0 es 1)
```

Solución:

```python
def potencia(base, exponente):
    if exponente == 0:
        return 1
    return base * potencia(base, exponente - 1)

print(potencia(2, 3))  # 8
print(potencia(5, 0))  # 1
```

Ejercicio R2: Invertir una cadena

Escribir una función recursiva `invertir(texto)` que devuelva el texto al revés.

```
invertir("hola") → "aloh"
invertir("") → ""
```

Solución:

```python
def invertir(texto):
    if len(texto) <= 1:
        return texto
    return invertir(texto[1:]) + texto[0]

print(invertir("hola"))  # "aloh"
```

Ejercicio R3: Fibonacci recursivo

Escribir una función recursiva `fib(n)` que devuelva el n-ésimo número de Fibonacci.

```
fib(0) → 0
fib(1) → 1
fib(6) → 8   (0, 1, 1, 2, 3, 5, 8)
```

Solución:

```python
def fib(n):
    if n == 0:
        return 0
    if n == 1:
        return 1
    return fib(n - 1) + fib(n - 2)

print(fib(6))  # 8
```

> **Nota**: esta implementación es ineficiente para valores grandes de `n` porque recalcula los mismos valores muchas veces. Para una versión eficiente, usar el generador con `yield` del ejercicio G2 o un bucle.

5. Ejercicios integradores (resueltos)

Estos ejercicios combinan las técnicas de toda la UD3: comprehensions, generadores, lambda, map/filter y recursión.

Integrador 1: Procesar notas — tres formas

> **CEs**: CE1_7, CE3_4, CE4_4

Dada una lista de notas, obtener las notas aprobadas (>= 5) redondeadas al entero más cercano.

```
notas = [3.7, 7.2, 4.8, 9.1, 5.0, 2.3, 6.6, 8.4]
```

Con comprehension:

```
resultado = [round(n) for n in notas if n >= 5]
# [7, 9, 5, 7, 8]
```

Con map + filter:

```
resultado = list(map(round, filter(lambda n: n >= 5, notas)))
# [7, 9, 5, 7, 8]
```

Con generador (si solo se necesita iterar una vez):

```
for nota in (round(n) for n in notas if n >= 5):
    print(nota)
```

Integrador 2: Pipeline de datos

> **CEs**: CE3_4, CE3_6, CE4_2, CE4_4

Dada una lista de productos con nombre y precio, obtener los nombres de los productos que cuestan más de 20€, en mayúsculas, ordenados por precio de mayor a menor.

```python
productos = [
    ("Camiseta", 25.0),
    ("Calcetines", 5.0),
    ("Pantalón", 40.0),
    ("Gorra", 12.0),
    ("Chaqueta", 80.0),
]
```

Con comprehension + sorted:

```python
caros = sorted(
    [(nombre.upper(), precio) for nombre, precio in productos if precio > 20],
    key=lambda p: p[1],
    reverse=True
)
nombres = [nombre for nombre, _ in caros]
# ['CHAQUETA', 'PANTALÓN', 'CAMISETA']
```

Con map + filter + sorted:

```python
caros = filter(lambda p: p[1] > 20, productos)
ordenados = sorted(caros, key=lambda p: p[1], reverse=True)
nombres = list(map(lambda p: p[0].upper(), ordenados))
# ['CHAQUETA', 'PANTALÓN', 'CAMISETA']
```

Integrador 3: Cuadrados impares — cuatro formas

> **CEs**: CE1_7, CE3_4, CE3_5

Obtener los cuadrados de los números impares del 1 al 20.

Con bucle for:

```python
resultado = []
for i in range(1, 21):
    if i % 2 != 0:
        resultado.append(i ** 2)
# [1, 9, 25, 49, 81, 121, 169, 225, 289, 361]
```

Con comprehension:

```
resultado = [i ** 2 for i in range(1, 21) if i % 2 != 0]
```

Con map + filter:

```
resultado = list(map(lambda i: i ** 2, filter(lambda i: i % 2 != 0, range(1, 21))))
```

Con generador (si solo se necesita la suma):

```
total = sum(i ** 2 for i in range(1, 21) if i % 2 != 0)
# 1330
```

Integrador 4: Factorial con recursión y generador

> **CEs**: CE4_3, CE4_5, CE1_7

Crear una tabla con los factoriales del 1 al 10 usando la función recursiva `factorial()` y una comprehension:

```python
def factorial(n):
    if n == 0:
        return 1
    return n * factorial(n - 1)

tabla = [f"{i}! = {factorial(i)}" for i in range(1, 11)]

for linea in tabla:
    print(linea)
# 1! = 1
# 2! = 2
# 3! = 6
# 4! = 24
# 5! = 120
# 6! = 720
# 7! = 5040
# 8! = 40320
# 9! = 362880
# 10! = 3628800
```

6. Tabla resumen

Comprehension vs generador

Aspecto	List comprehension	Generador
Sintaxis	`[expr for x in seq]`	`(expr for x in seq)`
Devuelve	`list`	`generator`
Memoria	Toda la lista de golpe	Un elemento a la vez
Reutilizable	Sí	No (se agota)
Soporta `len()` , indexación	Sí	No

Lambda vs def

Aspecto	`lambda`	`def`
Nombre	Anónima	Con nombre
Cuerpo	Una sola expresión	Múltiples líneas
`return`	Implícito	Explícito
Uso típico	Argumento de `sorted`, `map`, `filter`	Función reutilizable

Equivalencias: comprehension vs map/filter

Tarea	Comprehension	map/filter
Transformar	`[f(x) for x in seq]`	`list(map(f, seq))`
Filtrar	`[x for x in seq if cond(x)]`	`list(filter(cond, seq))`
Ambas	`[f(x) for x in seq if cond(x)]`	`list(map(f, filter(cond, seq)))`

Recursión vs iteración

Aspecto	Recursión	Bucle
Estructura	Caso base + caso recursivo	Condición + cuerpo
Memoria	Más (pila de llamadas)	Menos
Mejor para	Problemas que se subdividen	Recorridos lineales